Where the capability plane stands, and what holds it up

docs/2026-10-18-capability-plane-thesis.md · built Mon 19 Oct 2026

Brief for the founder, 2026-10-18. It answers your question about the capability plane, checks the scout's sweep against the changelog, and reviews deck v3. The sources and abbreviations are listed at the end.

1. The answer in five sentences

1. The capability plane is the half of Felix that does three things. It decides what engineering a project owes and keeps that list standing across sessions, hosts and models. It supplies each item (a probe, a qualified checker, a tool, a worker) under a recorded lease no wider than the need. And it keeps or retires each supplier on outcomes that the supplier did not grade.
2. It is not secondary, because the trust plane can only judge what someone listed. The failure Felix was built for was a listing failure: a deploy shipped green with no rate limiting, and three separate authors missed it. The trust plane's own stated residual is that "one authority still writes both the work and the judgement of the work" (amendments §6), and part of closing it is the capability plane deciding who writes what.
3. Felix rents the labor and every mechanism hosts are paid to improve: models, worker runtimes, fork and background agents, picking the cheapest model above a floor, the agent console, the catalogues, and the model that does the noticing.
4. Felix owns what a host is not placed to hold:
   - the grounded, standing need ledger and its denominator;
   - each need's supply state;
   - the lease and its record;
   - the consequence floor;
   - the order and isolation of units of work (v3.2's Execution Coordinator);
   - when a capability appears;
   - retirement on attributed outcomes.
5. One result would prove this account wrong. Suppose that on the sealed seeded-omission corpus, a plain session on the strongest available host model, with the host's memory and side agent switched on, finds the planted omissions and still acts on them after a restart as reliably as Felix's ledger holds them. Then standing, grounded intake adds nothing over the host. Under roadmap §5 as amended on 10-01, that result would be investigated before anything narrows.

The seven invariants, one line each (section 4 argues each one):

	Invariant	Felix
I1	A need is standing, grounded in observed or declared facts, and printed with its denominator	builds
I2	Every admitted blocking need has a supply state: a qualified instrument, a build under lease, or a person's route	builds
I3	Every worker runs under a lease Felix computes before launch; the host enforces it	owns the contract, rents enforcement

	Invariant	Felix
I4	Assurance is set by consequence, never by difficulty, and no verdict reads which model did the work	owns the contract
I5	The capability plane allocates independence (who writes the check), and the lease records it	owns the contract, builds the record
I6	A provider keeps its place on outcomes it did not grade, and attribution flows from trust to capability only	builds
I7	Recorded project state, not prompt keywords, decides when a capability appears	builds

Why it feels secondary. It is the plane with no measured number yet. The trust plane has failure data behind it. On capability, 22 obligations have been proposed and 0 admitted, and the seeded-omission corpus that would give it a number has had a settled protocol since 10-01 and has not run. The idea is not the weak part. The missing evidence is.

2. The scout, checked

All three verifiers read the full changelog, 9,197 lines. Line numbers below refer to that file. It has no 2.1.297 or 2.1.298 heading. The whole 2.1.303 entry is "Bug fixes and reliability improvements" (line 5).

Claim	Verdict	Version	Correction
S1: checked through 2.1.303, released Oct 18	Partly	2.1.303	2.1.303 is the newest heading, but it documents no changes, and the changelog carries no dates. Everything relevant is in 2.1.300 to 2.1.302.
S2: agent.spawn can rewrite a workflow agent's prompt and model	Confirmed	2.1.300, line 135	Workflow agents first reached agent.spawn in 2.1.292 (line 710), refuse-only. The changelog only declares the rewrite. It says nothing on what runs when the rewritten model is lower, unavailable or restricted.
S3: tool.check "now" carries agentId	Partly	2.1.290, line 806	Real but not recent: it is thirteen releases old, and it covers plugin (mod) hooks only. It is measurable on the builds installed today.
S4: forks on by default in -p, SDK, Desktop, VS Code; main-conversation subagents backgrounded	Confirmed	2.1.302, lines 39-40	The scout left out the second half of line 40: a subagent's own Agent calls now run in the foreground by default. `CLAUDE_CODE_FORK_SUBAGENT=0` opts out.
S5: agent-hook model falls back to the caller, so "Felix's minimum is no minimum"	Partly; the inference is wrong	2.1.300, line 156	The line is about hooks of type `agent`, whose verifier model now falls back instead of allowing the action. It does not mention agent.spawn. The real silent-downgrade paths are line 1237 (2.1.286: a refused model retries once on the previous model of the same tier) and line 82 (2.1.301: `fable` and `best` retry on the previous Fable).
S6: agents view, `← for agents`, controls, notifications, transcripts, filters, survival, `$.ui.view`	Mostly old or misdated	2.1.139 to 2.1.300	The agents view dates from 2.1.139 (line 5548). The `←` hint is older still, and 2.1.302 hides it after 10 seconds (line 41), so S6b is wrong. Model and effort controls set new-session defaults (2.1.299, line 228). Live changes came in 2.1.290 (line 960). Filters arrived in 2.1.287-288. No line in the window says background agents survive the parent turn. `$.ui.view({agentId})` is real (2.1.300, line 131) but only displays a transcript. All of it is human UI, not worker semantics.

Claim	Verdict	Version	Correction
S7: "You should know" side agent; "Anthropic's advisor"	Partly	2.1.287, line 1121; advisor from 2.1.117	These are two separate things. "You should know" is a built-in mod. It is off by default, needs telemetry on, and only flags. `/advisor` is a server-side API tool that runs a chosen second model. Neither is a hook or a gate.
S8: changelog full of worker reliability fixes	Confirmed	2.1.293 to 2.1.302	Three entries bear on Felix: line 85, line 338 and lines 341/533 (see below).
S9: open issue #87874 on join, cancellation and quiescence	Partly	not in the changelog	It is open as fetched, but it is one user's design request, read through a summarizing fetch tool. No changelog line touches it.
S10: 2.1.300 makes the advisory floor "plausibly enforceable"	Partly	2.1.300 line 135; 2.1.296 line 415	0cw's measured reason came from a PreToolUse updatedInput rewrite on 2.1.284, which lowered the model (0cw, Wave 1). 2.1.300 adds a different mechanism for a different class of agent, and it does not answer that measurement. "Plausibly enforceable" is the scout's phrase, not the queue's.

What the scout missed that matters

Change	Where	Why it matters to Felix
`onFailure: "block"` for command and HTTP hooks	2.1.295, line 495	Felix's own command hooks can be declared fail-closed without becoming mods. That removes one of the two reasons 0cw item 8 gives for declining an input-rewriting floor ("adds an enforcing output to the fail-open hook"). It is unmeasured: it is not yet known which events honour it, whether a plugin's hooks.json may set it, or what happens on timeout. A blocking failure on Stop could trap a session.
`CLAUDE_CODE_WORKFLOW_SUBAGENT_MODEL`	2.1.296, line 415	A non-hook lever that pins every workflow agent to one model. It is a pin, not a floor, and it can lower as well as raise. The installed CLI 2.1.296 has it, so it can be measured today on the stand-in. One open question: can a project's settings `env` set it, which would let a repository lower the model?
Workflow agents skipping a step while the run still reports success	2.1.301, line 85	Act now. Every Felix Workflow pipeline runs on desktop 2.1.295 or CLI 2.1.296, where this bug is live. Until 2.1.301 or later is installed, a green Workflow result must be checked stage by stage against each stage's artifact.
`updatedInput` validated like Claude's own input	2.1.300, line 193	Changes the mechanism behind the measured lowering. The 2.1.284 result has to be re-run on 2.1.300 or later before anyone cites it again.
agent.spawn did not run on resumed subagents	fixed 2.1.299, line 338	On the installed builds, a floor or lease on agent.spawn misses every resume.
Agent tool takes `effort`	2.1.292, line 706	The floor in 0cw item 8 is "model, effort and verification depth". Effort is now a per-call input that can be raised or lowered.
`UserPromptSubmit` runs before skill expansion	2.1.299, line 386	The router's input changes: it now sees the unexpanded slash command. Misrouting corpora have to name their build.
A mod can no longer push a headless run past `--max-budget-usd`	fixed 2.1.299, line 239	Adds to the cap already measured soft (39 of 39 capped runs exited 0 over their cap; 0cw, Wave 1).
Plugin-hook delivery failures now fail closed	2.1.299, line 370	Felix's hooks ship in a plugin. It is not yet known whether this covers command hooks in hooks.json.

What is installed, as read on 2026-10-18: CLI 2.1.296, and desktop 2.1.293 and 2.1.295 (this session runs 2.1.295). Nothing at 2.1.300 or later is installed. The 2.1.286 desktop build that #417 names as a measurement arm is no longer under Application Support.

Net effect on the queue

Nearly all of this goes into the mods round (#417, specified and not run):

- Mark the Q4 premise ("agent.spawn can only refuse a Workflow agent") stale from 2.1.300.
- Split Q3 into `turn.start` and `turn.step`. `turn.start` hears subagents only from 2.1.299 (line 388).
- Add effort, fork and resume cases to Q5.
- Split Q7 by build.
- Add six questions. All of them can run with no tokens, and each has an arm on the installed builds:
  - Q9: does `onFailure: "block"` make Felix's own command hooks fail closed, on which events, and what happens on timeout;
  - Q10: the workflow-model environment variable, and whether a project's settings can set it;
  - Q11: whether any Felix hook answers "allow", and if so whether plan mode runs the write on builds before 2.1.299 (line 341);
  - Q12: step-skipping on a Felix-shaped script;
  - Q13: what runs when the floor's named model is refused (the corrected form of the scout's S5);
  - Q14: the router's input before and after 2.1.299 moved skill expansion after the hook (line 386).

The full per-question rewording is in `verification.json` (field `mods_round_delta`) in the memory root's `audits/2026-10-18-capability-plane/`.

A 2.1.300+ arm needs a binary downloaded into the rig folder, not installed in place. `claude install` would re-point your live CLI symlink.

Outside #417:

- The Workflow stage check above starts now.
- 0cw item 3 still says "narrow to the obligations people name". That contradicts your 10-01 amendment in the same file and in roadmap §5. README.md:410-414 says the same. Both get fixed in the next handoff fold and README pass.
- R8 does not gate lane item 4. The v0 lease has no Agent tool, so it cannot fork, and the dry run launches nothing. R8 does gate the D3 tick, which must re-check its flags on the binary it actually runs.

3. Why it felt lost

Three documents put the capability plane on the layer hosts are eating, and each did it in a different way.

The deck gives capability to the model. Slide 2 lists "Choose tools. Spawn specialists." on the side the AI stays free to do. Slide 11 puts "chooses tools / spawns workers" under probabilistic cognition. The constitution says the opposite: Felix owns "which MCP, which plugin, which skill, which agent, in which order, in one context or several". The deck then:

- states the trust plane as invariants (slides 9 to 12) but the capability plane as open questions and a future (slides 13, 14, 17);
- titles slide 8 "The failure is not capability. It is self-certification";

- lists six failure classes on slide 5, none of which is omission.

Your founding failure is missing from the deck. Without it, the deck can argue only for the trust plane, and capability reads as a speed feature. Slide 13's question "How well did they perform?" points toward ranking workers. Slide 15's "Coordinator: plan + integrate" is the crowded shape (Factory Missions, Claude Code Projects, Harness Cosmos per the market map).

The scout looks only at the worker runtime. Every one of S2 to S9 is about how workers spawn, fork, background, report and get rendered. Those are supply-side mechanics that Anthropic ships weekly. R1 ("Felix as scheduler/policy/acceptance layer") is the narrowing you rejected on 10-10.

Your "sharpest feature" is on the supply side. Evaluating difficulty and deploying workers to match is labor allocation. D1 (0cw, 10-01) had already rented the part that minimizes models, so losing it to Factory costs nothing Felix kept. Your 09-02 intent was demand-side: "what did neither know to look for" (felix-is-an-omission-engine). The market map points the same way. After the 09-22 delta, derivation inside one run is occupied. What stays empty:

- a derived UNKNOWN held blocking in a standing ledger;
- receipts that go stale on inputs outside the tree;
- retiring a capability on observed value;
- independence from the vendor being judged.

The answer is already in the deck's slogan. Slides 1 and 2 say: "Do not make the AI deterministic. Make the boundaries deterministic." The deck applies this only to evidence and release. Apply it to capability as well.

The free side is the host's and improves every release:

- decomposing a task;
- picking a procedure;
- spawning helpers inside a task;
- choosing the cheapest model above a floor.

The deterministic boundary around capability is Felix's:

- what the project owes;
- which capabilities are present, and under what lease;
- the minimum assurance a consequence demands;
- who may write the check;
- what counts as a capability having served;
- when a capability is retired.

Your own architecture already says this. v3.2's fourth non-negotiable invariant is the requirement and method split: "Felix owns what must become true. The host owns how to make it true and may propose equivalent evidence." Read precisely, "how" is the method inside one unit of work. It is not which units of work exist, their order, or their isolation. Those stay with Felix's Execution Coordinator and with the constitution's "which agent, in which order". Factory's difficulty routing and Anthropic's fork, background and agent.spawn features all sit on the method side of that line. That is why losing them costs Felix nothing it was meant to own.

One refinement keeps this inside v3.2. The capability plane proposes and provisions. Admission, the Evidence Kernel and Release decide what binds. That is one boundary principle with two authorities and separate stores (v3.2 §1, §12).

4. The invariants

Seven survived refutation or were repaired. Each restates or sharpens text already in v3.2 and its amendments. None is a new kernel. Candidate keys: **A** model-invariance, **B** host-absorption, **C** founder-intent, **D** user-experience, **E** main-session. The five were written blind to one another, so convergence is evidence.

I1. A need is standing, grounded, and carries its denominator

Statement. Every enforcing row in the need ledger cites OBSERVED or DECLARED grounding. That means topology from probes Felix builds and pins, the declared risk envelope, or intent the person stated, recorded with provenance (v3.2 §4, §8). Any source may propose a need:

- a Discovery Worker;
- the host's side agent;
- ECC or another baseline provider;
- a scanner;
- a person;
- the prompt.

No proposal grounds itself, and a single probabilistic proposal stays advisory (v3.2 §4, pathway 4). The ledger outlives any session, host or model. A row leaves force in only four ways:

- derived dormancy;
- a policy version change;
- a committed supersession;
- for TASK_BLOCKING rows only, a scoped, expiring exception (amendments §3).

Every "N unresolved" prints "of M, residual R". The engine computes the residual against a named, versioned list of situation classes. The proposer never reports it, because a model's report about its own blind spots "is worth nothing" (amendments §2.2).

Why it survives a stronger model. A better model notices more within a turn. That does not make a noticed need survive the session's end or a change of agent. Topology also changes while no model runs: an endpoint is removed, a policy changes, a dependency is bumped.

Why a host will not own it. Factory derives a validation contract within one mission. That contract is user-skippable, not commit-bound, and nothing in it goes stale (market map, 09-22). "You should know" is advisory and in-session. Hosts can and will ship a standing advisory list (Projects shared memory, Factory's 08-27 post). What they are badly placed to ship is a list that blocks, is grounded in probes, and is held by a party independent of the vendor being judged.

Falsifier. Three tests:

- (a) Structurally, an enforcing row whose grounds field is `–` violates I1.

- (b) The north-star eval: an obligation admitted earlier blocks an incomplete deploy in a session that never mentions it, then goes dormant through topology.
- (c) The sealed corpus with a second-family arm and a strongest-host arm. The host arm needs a protocol amendment, because the settled protocol runs `--bare` with telemetry off, and "You should know" requires telemetry on (line 1121).

Class. BUILD the ledger, grounding, residual and request protocol. RENT the proposing model.

Today. Built:

- probes.sh and facts.sh;
- the request and answer protocol (candidates.sh, commission-provider.sh);
- 22 candidates proposed (`~/.felix/memory/elenchus/candidates.tsv` has 7 rows, `~/.felix/memory/veldra/candidates.tsv` has 15).

Not yet:

- No candidate has been admitted. The only admitted rows are felix-home's two deterministic ones, and Elenchus holds only `lifecycle_wired`.
- The ledger lives in felix-home and the memory root until Phase 2.
- Nothing schedules discovery (amendments §1.3).
- No residual is printed.

Converged. A (CP1, CP6), B (C1, C2), C (CP-1), D (CP1, CP6), E (C1, C2): five of five.

I2. Every admitted blocking need has a supply state

Statement. Each admitted blocking obligation has exactly one of three supply states:

- a qualified instrument, meaning a probe or checker that met all seven controls and was qualified by a party other than the worker;
- a named acquisition or build in progress, under its lease;
- an explicit HUMAN_ATTESTATION or AUTHORITY_BLOCKING route (v3.2 §11, §4).

Without one, it is reported as an unsupplied capability need, and the trust plane holds it. Qualifying a checker is trust work (amendments §2). Supplying the checker is capability work. A check that enforces on fewer than seven controls is listed as an exception with an owner and is never cited as conformance.

Why it survives a stronger model. A checker that cannot fail is worthless whoever wrote it. Qualification evidence also does not transfer: a fresh model that regenerates a probe does not inherit the old probe's discrimination evidence. So the accumulating asset is the evidence, not the script.

Why a host will not own it. A same-vendor model grader is a MODEL_CLAIM (v3.2 §11). A non-model judge whose instruments carry seeded-failure evidence for this repository is not something a vendor grading its own agent will ship.

Falsifier. Either of these violates I2:

- an admitted blocking row with no supply state that reports nothing;
- a qualified instrument that passes a seeded violation.

Class. BUILD. `capabilities_needed` is Phase 1 trust-track work in obligations.sh. The capability lane supplies the instruments.

Today. The `needs` column is REPORT ONLY (engine/harness/lib/obligations.sh:39). Every candidate's checker was found flawed on blind re-check (0cw, founder's open items). The `independence` check enforces with 4 of 6 controls, and whether it is grandfathered is still open.

Converged. A (lane 3 and plane relation), C (CP-2), D (CP2), E (C6 repaired): four of five.

I3. Least authority by lease: Felix computes it, the host enforces it

Statement. Every worker Felix dispatches launches under a lease that Felix computes deterministically before launch. The inputs are the need's enforcement class and the risk and channel tables. A model may only narrow a lease. The lease names:

- the toolbox;
- writable paths;
- the pool;
- the full model id;
- `claude --version`;
- expiry;
- a spend limit.

Fields the host enforces at launch cannot be widened from inside. Spend is soft on every host measured, so it is checked after the run and any overshoot is recorded as a breach. Any field the host does not enforce is reported as "unenforced", never as a bound. Installed providers are admitted at a v3.2 §7 trust tier and loaded only into launches whose recorded need names them. A provider the host cannot scope inside a session is recorded as "unscoped" and never runs release-critical work or checker authorship. Acquiring or activating a capability never silently expands production authority (v3.2 invariant 5). Output enters the trust plane only as an answer file read offline.

Why it survives a stronger model. A more capable worker holding broad authority has a larger blast radius. Channels cannot be earned (docs/2026-08-15-authority-that-cannot-be-earned.md).

Why a host will not own it. The mechanisms are arriving and are rented:

- `--restricted` and `--strict-mcp-config`;
- tool.check agentId (line 806);
- the agent.spawn rewrite (line 135);
- NVIDIA OpenShell (market map, 10-10).

Computing a lease's breadth from a repository-owned need, and recording it in a form that survives a change of host, is not part of any of them.

Falsifier. The forgery test. A scripted worker in an isolated lease attempts a write outside the lease, a trust-plane write, and a tool call outside its list. Any one success breaks I3.

Class. OWN_CONTRACT. RENT enforcement.

Today. The v0 shape is decided and its flags were measured on 10-01 (direction-decisions §3; 0cw item 4). admit.sh refuses risk=high, and discover.sh reads a candidate at its pinned sha. Not built: the dry run (`felix` has no dispatch subcommand), breadth computed from obligations, the sandbox, and revocation.

Converged. A (CP4), B (C3), C (CP-3), D (CP3), E (C3, C5): five of five.

I4. Assurance is set by consequence, and no verdict reads the model

Statement. The assurance a unit of work requires covers discovery angles, verification depth, independence class, and whether a human must authorize. It is set by three things:

- the unit's consequence class (v3.2 §10: exposure, authority, reversibility, money or data, novelty, production proximity, historical miss rate);
- its non-delegable channels;
- the record of accepted outcomes (D1).

Task difficulty and worker identity never lower it. Under D1, Felix also publishes a producer floor: a minimum model and effort, written as full model ids, never as aliases. That floor is advisory and raise-only (0cw item 8), and the host picks the cheapest worker above it. No receipt, merge blocker or release transition reads which model did the work. A separation-of-duties check may read whether the checker's author and the worker are the same party, but never what either one is. Leases and outcome records name the full model id as provenance only.

Why it survives a stronger model. Difficulty is a property of task and worker, and it falls every generation. Consequence is a property of the change and the world, and it does not. When labor gets cheap, assurance becomes the scarce thing. Aliases also step down silently (lines 82, 1237), which is why ids are written in full.

Why a host will not own it. GitHub, Cursor and Factory ship model minimizers (D1). None ties a floor to a risk table the repository owns, and a vendor routing its own models has no incentive to.

Falsifier. The replay's predeclared rule settles the model part (0cw item 2). Any read of a model id or `resolvedModel` on the enforcing path violates I4.

Class. OWN_CONTRACT.

Today. risk.tsv tiers (path-based) and channels exist. Budget classes, triggers and the stop rule are roadmap §4, not before March. The floor is unbuilt.

Converged. A (CP2), B (C4), C (CP-4), D (CP4), E (C5): five of five. This was also the candidates' most repaired invariant. Three versions either let the floor decide acceptance or let a model change invalidate receipts, and both break trust invariance.

I5. The capability plane allocates independence, and the lease records it

Statement. When an obligation's enforcement class and budget class (B2 and up) call for independent evidence, the Capability Manager provisions three separate leases: the checker author, the mutation author and the verifier. Their readable inputs are disjoint. The mutation author is denied Read on the checker path only, because control 5 needs it to see the artifact's markers. The Evidence Kernel checks blindness from the lease records, never from model names. Cross-family provisioning for B3 and B4 is capability-side policy

written on the lease, never a receipt condition. Until the record is judged from outside the tree it attests (the judge pin, Phase 0b step 15), it is evidence of intent, not proof: state files are unsigned and any same-user process can write them (felix-floor-and-wall).

Why it survives a stronger model. Competence is not evidence that a checker discriminates. The four basic controls plus a behavioural witness passed the strongest cheat 8 times in 8, and the decoupled controls caught it (amendments §2). That measurement was of control design, not of model generations.

Why a host will not own it. The vendor paid when its agent finishes cannot credibly audit that agent, least of all with a competitor's model (market map, incumbent lens). Cross-vendor work also has to run on the customer's own keys (agent-vendor-access-and-terms).

Falsifier. Blind same-family authors against blind cross-family authors, under all seven controls. A separate arm puts same-context authors against leased blind authors on the qualification suite.

Class. OWN_CONTRACT, plus BUILD the lease record.

Today. Blind authorship is a build-pipeline convention. Control 7 is absent. Codex read-only as a second blind author is decided (roadmap §3, lane item 4). This is a real route to the amendments §6 residual, though not the only one: §2.4's trust-side discrimination evidence is another.

Converged. A (CP3), B (C4), C (plane relation), D (plane relation), E (C4): five of five.

I6. A provider keeps its place on attributed outcomes, and attribution flows one way

Statement. A provider with standing authority keeps that authority only while something justifies it. Standing authority means:

- hooks that run in every session;
- a credentialed MCP server;
- an auto-loaded skill set;
- a worker class.

The justification can be:

- an open obligation;
- a recorded labor need;
- a measured contribution to accepted outcomes in the attribution record.

Attribution is labelled with a lag: no revert, no later fix to its lines, no reopened obligation within the window. This is the 08-14 rule that "success needs a lag". Each label names the checker set it was judged under and carries a same-authority mark until the pin. Post-deploy incidents count. Below a stated sample size the verdict is UNKNOWN, and hook-only providers are reported as unmeasurable. Retirement is proposed, never silent, and is keyed on what was installed, never on what a route named. Attribution flows from trust to capability only. When a provider whose output is evidence (a probe or a checker) changes its pinned state, the receipts built on that output go stale (v3.2 §7, step 7). A worker's model is not such a provider.

Why it survives a stronger model. Better models produce more providers, and generic benchmarks say less about one particular repository.

Why a host will not own it. Disuse reporting exists: unused-plugin reports, `claude plugin eval`, Tessl's with/without evals. Outcome-keyed retention per repository, across competing vendors, does not.

Falsifier. Keep two lists for a quarter on two ecosystems, with a minimum sample: one keyed on outcomes, one keyed on disuse. If they agree, outcome keying adds nothing. Separately, any receipt code path that reads the provider field breaks trust invariance.

Class. BUILD.

Today. `felix_ledger_retire` is route-keyed: `_felix_ledger_zero_rows` and the `named` count at engine/harness/lib/ledger.sh:234-255. Lane item 7 is pending. Nothing reads `resolvedModel`.

Converged. A (CP5), B (C5), C (CP-2), D (CP3 exit), E (C4): five of five.

I7. Recorded state, not keywords, decides when a capability appears

Statement. A discovery pass, an activation or a dispatch happens when one of these recorded state changes occurs:

- a new decision epoch;
- a new surface in the topology;
- a raised risk or budget class;
- a recorded intent change, meaning a decision row with provenance written from what the person said;
- an UNKNOWN entering a release window;
- a receipt going stale.

Every trigger is recorded, and so is every check that fired nothing. Until the discovery schedule exists (roadmap §4), triggers are computed and shown, and a person's tick fires them. This makes your 09-02 "mid session, early session, late session" checkable.

Why it survives a stronger model. Topology changes while no session runs. A stronger model in session still cannot see another repository's ledger or a receipt gone stale outside the tree.

Why a host will not own it. Host suggestions are matched to the session's context. Firing on a project-owned ledger is not.

Falsifier. Re-measure with the blind method used on Aug 30, against a precision threshold declared before the run.

Class. BUILD the computation. Firing it unattended needs the schedule grant (amendments §1.3).

Today. The keyword router decides. Its measured precision is about 2% (n=158). It was reached 24 times in 418 sessions, and 106 declines are on record (misrouting-belongs-in-the-capability-queue).

Converged. B (C2), C (CP-5), D (CP1), E (C1), and A in part (situation triggers in CP6): four to five.

Invariant	A	B	C	D	E	Class
I1 Standing grounded need with a denominator	yes	yes	yes	yes	yes	BUILD
I2 Supply state per blocking need	yes		yes	yes	yes	BUILD
I3 Lease computed by Felix, enforcement rented	yes	yes	yes	yes	yes	OWN, RENT enforcement
I4 Assurance by consequence, model-blind verdicts	yes	yes	yes	yes	yes	OWN
I5 Independence allocated and recorded	yes	yes	yes	yes	yes	OWN, BUILD record
I6 Retention on attributed outcomes	yes	yes	yes	yes	yes	BUILD
I7 State triggers, not keywords	part	yes	yes	yes	yes	BUILD

Two proposals were moved out of this list. "Return only the irreducible" (all five candidates) is a property of the whole product, so it is the autonomy measure in section 5. "No output claims completeness" (A CP6, D CP6) is mostly trust-plane work under amendments §6. Its capability-side part is the residual in I1.

5. What "truly autonomous" means

Autonomy here is not "no human". It is engineering leakage per objective driven to zero, while correctness guards hold.

Leakage is counted from the session record against the seven leakage kinds in docs/episodes/method/codebook.md:

- mechanics-correction;
- repeat-settled;
- answered-agent-question;
- caught-unverified-claim;
- unstuck-needless-stop;
- translated-to-engineering;
- caught-overreach.

"Other" covers mostly typing commands the agent handed over. Two further cases also count: a hold Felix typed ENGINEERING, and a need a person named that an OBSERVED fact Felix already held should have produced.

A legitimate return to the person carries one of these types, recorded when the return is raised:

- product intent or taste;
- an architecture choice the person reserves;
- a risk-envelope fact no probe can observe (v3.2 §4, §8);
- an authority grant or channel act;
- an irreversible act;
- admitting an obligation, merge consent, or a direction approval, each of which is a person's commit by design;
- an attestation no instrument can supply;
- a critical UNKNOWN that its assigned budget class could not resolve, reported with that class.

A person may always add a need. It enters through the person pathway and is not a defect, because "expert and novice use one system and differ only in the resolution of the constraints they supply" (felix-endgame-control-plane).

The guards, without which a Felix that asks nothing and blocks nothing would score perfectly:

- recall on the sealed seeded-omission corpus at or above every control arm;
- the false-block rate in real sessions under a bound declared before measuring;
- stale receipts accepted = 0;
- product defaults the person later reversed, counted;
- valid obligations autonomously discharged, counted (v3.2 §14);
- the residual printed beside every verdict.

"**Truly autonomous**" for a project at a given risk class means leakage = 0 over a fixed window, every return typed, and every guard holding. It is bounded by design: channels and irreversible choices never become autonomous (constitution invariant 3). What runs unattended may widen only within the reversible envelope, and only as far as qualified, lagged evidence covers it. The driver of that loop is v3.2's Execution Coordinator, which Felix keeps and does not rent.

Today none of this is computed. The codebook exists, but its two readers agreed on only 86 of 123 turn labels (70%) in the second batch (docs/2026-09-23-episode-batch-2.md:133), and nothing types returns. On the capability side Felix performs no unattended act, because no dispatch exists.

6. The anti-slop contract

The line between Felix and Base44-style "idea to production" is not speed. It is whether the end-of-run claim has a denominator the builder did not write, and whether that claim stays true after the run. At the end of an objective the user should be able to check each of these:

#	What the user checks	Today
1	**Owed.** Every obligation, each with a plain reason grounded in an observed fact, its class, and its state (satisfied, UNKNOWN naming what would resolve it, or dormant with the grounds), including ones the user never named.	Not built: no admitted derived row exists anywhere. The candidate protocol is built.
2	**Proven on this exact thing.** Each satisfied item carries a receipt bound to the revision, and to the deploy config where one exists. Older receipts read STALE, never green.	Tree-bound receipts and tree staleness are built. Deploy-config staleness is Phase 2, unbuilt.
3	**The checker could fail.** Each instrument shows its qualification and who wrote it, next to who did the work.	Built for Felix's own checks. Arming for other projects waits on control 7 and the pin.
4	**What was not looked at.** Examined classes, the budget, and the residual, with "could not look" listed apart from "looked, found nothing".	UNKNOWN holds, on felix-home only. The residual is unbuilt.
5	**Decisions.** Every product, risk and authority decision that was asked is marked as the user's. Defaults Felix took are listed as overturnable, and anything that changes the risk envelope was asked.	Not built.
6	**What was installed and allowed.** Each activated capability with its tier, scope, reason and expiry. Nothing high-risk ran without a person.	The risk=high refusal and read-before-install are built. Leases and need traces are not.
7	**A stranger can replay it.** Ship or hold is recomputed from the repository alone, offline, with no model and no vendor.	Not built (Phase 2 and Phase 4).
8	**It stays true.** The next change reopens exactly the obligations it touches.	In-tree only.

The headline line reads: "live URL; N decisions you made; M obligations proven on this revision, of a stated denominator; K UNKNOWN holding; residual R". The verdict vocabulary of v3.2 §11 (SHIP, HOLD, ESCALATE) is the target, and none of those words exists in the engine today. What exists is the Stop hold and the gate's pass or fail. The words Felix never uses are "production-ready", "one prompt" and "no engineering needed". Outside text claims only rows marked built. Today that is item 2 in part, item 3 on Felix's own checks, and the hold half of item 4 on felix-home.

7. What Fable 5.5-class models change and do not change

The changelog has Fable 5 as "a Mythos-class model" (2.1.170, line 4901) and Fable 5.1 as the default Fable at $10/$50 per Mtok (line 3028). Fable has a weekly limit and usage-credit gating (lines 1576, 1671). Fable 5 is offered as an `/advisor` (line 3702). The `fable` and `best` aliases retry on the previous Fable when the newest is unavailable (2.1.301, line 82). No Fable 5.5 appears through 2.1.303, so what follows treats 5.5 as the next tier up.

What changes:

- In-session noticing improves. That is a dividend, since Felix rents the proposer.
- The control arm improves too. The corpus has to be re-run each generation with pinned full model ids, and it should carry a strongest-available-model arm. Both the replay rule (0cw item 2) and the corpus currently compare only Sonnet and Opus.
- Labor gets cheaper, so the model part of the floor may fall toward "any". The replay's predeclared rule decides that.
- Hosts ship more mechanism, so the RENT column grows. Each Felix mechanism that duplicates a host feature becomes a deletion candidate, while the responsibility stays with Felix.
- Fable's price and gating make a floor written as an alias more dangerous. The alias can step down silently, and a floor that names Fable has a cost that D1's "cheapest above the floor" must see.

What does not change:

- A session ends, and a project's needs persist.
- The party paid to finish cannot be its own denominator or its own judge (v3.2 invariant 1).
- Competence does not earn a channel.
- Grounded state changes while no model runs.
- Risk-envelope facts are product intent that no model can observe.
- A model cannot certify its own residual.
- More capable models produce more providers, which makes admission, leases and retirement matter more.

The net effect: Felix builds less mechanism and owns more contract. If a Fable-class plain session matches Felix on the corpus, that is a finding to investigate under roadmap §5, not a scope change.

8. Build, own, rent, drop

Item	Verdict	Decision and reason
0cw 1: host measurement	MEASURE_FIRST	Fold in S2-S5 and the misses above as the #417 round, with Q9-Q14 added. No tokens; it can run on the installed builds today.
0cw 2: held-out replay	MEASURE_FIRST, as designed	Founder-approved, with a predeclared rule. Candidates that re-scoped it in advance were overruled. Under D1 a minimizer outcome goes to the host, while an escalation or adaptive outcome is Felix's capability-lifecycle policy.
0cw 3: seeded-omission corpus	BUILD, first	It is I1's falsifier. Add the second-family and strongest-host arms by protocol amendment, and fix the stale "narrow" text. It needs your spending ceiling.

Item	Verdict	Decision and reason
0cw 4: `felix dispatch --dry-run`	OWN_CONTRACT	It is I3's unit. Not gated by R8. Point the first lease at an obligations request, so dispatch starts as a Discovery Worker.
0cw 5: attended tick (D3)	BUILD	Item 4 must land first (#350 has merged). It re-checks flags on the binary it runs. The key is your act.
0cw 6: outcome record, then passive routing	BUILD, both stages	It is I6's substrate. Keep passive routing, because M1 requires "one measured routing decision". Record only; it never touches verify.sh or receipt.sh.
0cw 7: retire list keyed to installs	BUILD	Precondition for Decision B. Waits on the ledger change on `claude/fresh-repo-tree-identity`.
0cw 8: consequence floor	OWN_CONTRACT, advisory	Full model ids only. The input-rewriting version stays declined. Q9 and Q13 report whether either of its two reasons still holds.
0cw 9: task-scoped checkpoint	MEASURE_FIRST, on its trigger	Conditional as 0cw already writes it: the replay keeps escalation, or M2's adaptive transition gets built. Not dropped.
Roadmap lane 4: Codex read-only	BUILD (run now)	The second-family author for I5 and the replay's Codex arm.
Roadmap lane 5: Codex adapter	OWN_CONTRACT	The test of host independence. About December, gated on its predeclared arm result.
R1: Felix as scheduler and acceptance, Claude Code as runtime	Split	RENT the runtime. OWN the Execution Coordinator: units of work, their order, isolation and lease. Reject the scope sentence, which narrows Felix.
R2: two worker classes behind adapters	OWN_CONTRACT	Class is assigned by consequence and by what the output becomes. Evidence work and release-critical work go isolated. Native workers sit under Felix's hooks, which fail open unless Q9 shows `onFailure: "block"` holds.
R3: reopen the advisory floor	MEASURE_FIRST	The measurement goes to you only if it changes what you would decide.
R4: per-worker leases on agentId	MEASURE_FIRST	Measurable now (2.1.290). It is a mod hook, so it collides with the "no plugin" lease. Native class only, and only ever a floor.
R5: no agent console; semantic overlay	Split	RENT transcript rendering. OWN the read-only data contract (M0, `felix status --tsv`). Anything drawn mid-session reopens the stealth decision.
R6: "You should know" or advisor as a capability	RENT as a Discovery Worker source	Never a judge (invariant 2). Its notes enter as single probabilistic candidates.
R7: native agents are not the source of truth	OWN_CONTRACT	Felix's lease row, result row and task key are the record. Line 85 and line 342 show why.
R8: rerun on 2.1.300-303 before the dry run	MEASURE_FIRST, re-scoped	Gates the tick and the native class, not the dry run.
Difficulty-based dynamic deployment	Split	RENT the minimizer (D1). The replay decides escalation and adaptive transition. OWN dispatch, composition and order. Felix's own form of deployment is by evidence: blind, cross-family and isolated as consequence requires (I4, I5).
Keyword router	Fix now, retire later	Stop routing input a person did not type (weekly batch). Re-measure with the Aug 30 method. Retire at Decision B after item 7, which is yours and not before March (roadmap §4). Do not improve the table.
Agent-console UI	RENT	The host's agents view renders workers. Felix builds M0's read-only pages.

Item	Verdict	Decision and reason
Provider discovery and admission	OWN_CONTRACT	RENT the listings (marketplaces, Tessl, AAIF). OWN pinning, read-before-execute, tier, lease and revoke. The sandbox (T2 is source-inspected) and revocation are unbuilt.
Obligation intake	BUILD	I1 and I2. The bottleneck is checker qualification, then admission.
Retirement on observed value	BUILD	I6, after items 6 and 7.
Mid-session mounting	Split	RENT the mechanism, which is a host flag. OWN when a capability appears (I7). Between sessions, the lease is the launch configuration.

9. The deck

These are recommendations. The deck is your document.

Slides 1 and 18. The arrow "trust → authority → capability" is accurate as build order for capability authority (v3.2 §13, step 5), but it reads as product order and as "eventually capability".

- Slide 1 subtitle: "From extreme AI use to deterministic boundaries for what engineering must exist and what counts as done."
- Replace the arrow with a loop: "need → supply → proof → release → outcome → need".
- Keep "Do not make the AI deterministic. Make the boundaries deterministic." and add under it: "Two boundaries: what must exist, and what counts."
- Slide 18's closing line: "It may be won by the system that knows what a company of them still owes, and lets you trust what they did."

Slide 2.

- Free side: "Reason. Explore. Decompose its task. Write code. Pick a procedure. Spawn helpers inside its task. Recover from failure."
- Deterministic side, added: "What this project owes. Which capabilities are present, under what lease. Who may write the check. Which units of work exist, and in what order."
- Mark the new items as being built.

Slides 5 and 8.

- Add "missed obligations" to slide 5's failure list.
- Retitle slide 8: "The failure is not competence. It is self-certification and omission."
- Add as failure #4: "Omission: a deploy shipped with green tests and no rate limiting. The same item was missed by three separate authors."
- The resume-sourced figures stay as they are.

Slide 7. Beside the trust questions, add: "Who decides what must exist before anyone writes a test? Who notices what neither you nor the model thought to check, and does it still block next week?"

Slides 9 and 10.

- Slide 9: present revision-bound proof as shared ground (AI-SDLC and GitHub stale approvals have it; market map). Lead with the extension: "a receipt also goes stale when an input outside the tree changes (planned)".

- Slide 10: "verified → checked by trusted base → merged → promoted" implies independent acceptance, which 0cw bars from outside text until the step-15 judge pin lands. Use: "Seeded-bug qualification runs on Felix's own checks today. Arming it for other projects waits on the last control and a judge pinned outside the tree it judges."

Slide 11.

- AI cognition: "decomposes its task, writes code, picks a procedure, spawns helpers inside its lease, reasons after failure".
- Felix, added: "which capabilities exist, under what lease".
- Under effective state, mark "workers may start/stop" as "(designed; dispatch not yet built)".
- Change "changes may be merged ... only when the contract is satisfied" to: "Felix's merge refuses without a current pass. A server-side required check enforces it on one of five governed repositories so far" (felix-floor-and-wall).

Slide 13.

- Retitle: "Two planes, one loop."
- Capability questions: "What does this project owe, derived from what was actually built and declared? Who may supply each item, under what lease? Who checks it, other than whoever built it? When does a provider lose its place?" Drop "How well did they perform?"
- Keep "capability without trust just makes the ungoverned system faster" and add: "Trust without capability is a gate over checks someone already wrote."
- The footnote depends on your decision in section 11. Recommended: "Capability authority (dispatch, leases) waits on trust gates; need intake is built alongside."

Slide 14.

- Intent: "Human supplies goal, taste and the important choices."
- Need: "Derived from observed topology and declared risk, never from the builder's checklist; held standing; blocks when critical."
- Split Provider in two. "Capability: admitted at a pinned version, under a lease no wider than the need." "Labor: rented from any agent, on any model above a consequence floor."
- Insert "Check: written by someone other than the builder" before Proof.
- Lifecycle: "Kept on accepted outcomes; retired when nothing it serves is open."
- Footnote: "Built: provider reading at a pinned version, the install policy, the need request protocol (22 proposed, 0 admitted). Not built: leases, dispatch, outcome-keyed retirement."

Slide 15.

- Replace "Coordinator: plan + integrate" with "Execution Coordinator: which units of work exist, in what order, under which lease and isolation; the host plans how inside each".
- Add a "Need ledger" box.
- Workers: "bounded tasks, deployed by what the evidence requires (blind, cross-family, isolated)".

Slide 16. Keep the "not implemented" label and add: "Felix itself governs engineering only."

Slide 17.

- Label "High-volume coordinator + worker workflow" and "Delegation is already dominant" as host properties. The jump came from Claude Code's Workflow tool (docs/2026-09-21-where-felix-stands.md:122).
- Current build focus: both planes, with capability's built and unbuilt lines from slide 14.
- Key proof: three numbers. Engineering leakage per objective, seeded-omission recall against plain Claude Code and a second model family, and the false-block rate.
- Replace "1-2 months" with the MVP milestones: M0 late October, M1 late November, M2 late January, all P50 (0cw MVP table).

10. Sequencing

The deck says "finish Trust Plane MVP before Capability work" (slide 13 footnote; slide 17). The roadmap has run a capability lane at one session in four since 09-25 (roadmap §3, 4c513055). The deck attributes its line to your plan as discussed in October, so this is a conflict for you to settle rather than drift for me to correct. My recommendation is the roadmap, for three reasons.

1. The MVP you defined on 10-01 already contains capability work. M1 includes the D3 tick and one measured routing decision. M2 includes the corpus against plain Claude Code and the replay's routing decision. "Trust MVP first" cannot be met by its own definition.
2. Trust-first has already stalled capability once. There were 0 capability-file commits from Sep 17 to 21 (capability-plane-stagnation-2026-09-25).
3. The north star needs an omission to block, and the one real omission found (Elenchus's spend ceiling) blocks nothing today because of an intake and instrument gap, not a trust gap.

The deck's line is right about one thing: capability **authority** (dispatch, leases, unattended acts) waits on trust gates, as v3.2 §13 orders it. The lane already respects that.

On 10-18 you said the original decision was a strong trust backbone first, with capability built on top. That holds for the part of capability that consumes trust verdicts: retiring on attributed outcomes (I6), a floor fed by accepted outcomes (I4), and giving a dispatched worker real authority (I3). It does not hold for intake (I1, I2), because intake is not on top of the trust plane. It feeds it. In v3.2 a Discovery Worker proposes and Admission admits (v3.2 §1, §4), so the trust plane can only judge obligations that intake produced. Without it, the backbone judges only checks someone already wrote: felix-home's two hand-written rows. The seeded corpus and the admission of the 22 candidates need no trust primitive at all.

The lane has not delivered at its share, though. From 09-25 to 10-19, 80 commits landed on main, and 16 touched capability-side libraries (discover, obligations, stack, ledger, adopt, toolbox, recommend, route, candidates, commission-provider, reach, reach-read, own, probes, facts, detect, intake). Two were features: declared tools' reach (#431) and discovery from an empty directory (#434). The other fourteen were fixes. None was a lane item. The binding constraints were the landing rate, the weekly usage budget, the rule against a new engine branch while more than 10 engine PRs are open, and the corpus's open spending ceiling. The share itself did not bind.

Decided here (lane order is Felix's per 0cw): a lane session counts only if it lands engine code or produces a sealed, measured number. The order is:

1. Workflow stage verification, now (line 85).
2. The extended #417 round on the installed builds, with no tokens.

3. The seeded corpus, with the added arms, once you set the ceiling.
4. Codex read-only, which is already allowed.
5. The dry-run lease.
6. The retire re-key, once its ledger change lands.
7. The outcome record.
8. The D3 tick.
9. The replay as approved.
10. The floor, after the outcome record.

Need-side items wait on no trust milestone. Supply-side items keep their trust gates.

11. What needs the founder

Decision 1: which order binds. Either the 09-25 lane, with both planes built together and capability authority gated by trust, or the deck's "Trust MVP before capability".

- If the lane binds, Felix's first capability act this quarter is a leased worker running an obligations request outside any session, with the corpus number beside it.
- If the deck binds, Felix stays a gate over checks a person wrote until about February, and the roadmap must say so.

Recommended: the lane. Re-word slide 13's footnote to match.

Decision 2: who admits a model-proposed obligation. Today only a person's commit admits (candidates.sh; direction-decisions §3), and the 22 candidates have waited since Sep 26-27. Two parts:

- (a) Decide the 22 now. Admitting one is how the omission engine first produces standing output. Their checkers were all found flawed, so admitted rows read UNKNOWN until an instrument qualifies (I2).
- (b) Whether, once control 7 and the judge pin exist, Felix may admit a witness-backed candidate itself (v3.2 pathway 2). The admission would be TASK_BLOCKING only, made by a Felix-authored commit the enforcing path can tell apart from yours. Release-blocking admission stays with you or with policy.

What changes about Felix: obligations nobody named would start blocking without you in the loop. The cost is false blocks, bounded by task scope. Recommended: decide (a) now, and say yes to (b) in principle with nothing armed before control 7 and the pin.

Consents, not decisions:

- the corpus spending ceiling (the protocol's estimate: $96-349 for one corpus at 3 runs, $172-632 with the replication; seeded-corpus `protocol.md:688–689`);
- a 2.1.300+ Claude Code binary downloaded into the rig folder for the #417 round, not installed in place.

Not asked now:

- The input-rewriting floor stays declined until #417's Q5, Q9 and Q13 show whether its two reasons still hold.
- Router retirement is Decision B, not before March.
- Whether the console should ever act stays read-only.

12. What was rejected and why

- "**Capability stays thin", or Felix as "scheduler, policy and acceptance" only (scout R1):** narrows Felix. You rejected it on 09-23, 10-01 and 10-10.
- "**Hosts decide how a task gets done", with scheduling rented:** right as v3.2 invariant 4 states it (the method inside a requirement), wrong if read as renting which units of work exist and their order. That would give up v3.2's Execution Coordinator and the constitution's "which agent, in which order".
- "**The noticing is rented, so noticing is not Felix's value":** the probes, the request, corroboration and the recall measurement are Felix's. Only the proposing model is rented.
- **Difficulty routing as a Felix-built minimizer:** D1.
- **Dropping difficulty or adaptive routing before the replay runs:** pre-empts an experiment you approved, with predeclared rules.
- "**The proposing model is never the producing model":** contradicts v3.2 §1 and §9, which allow host reasoning as a Discovery Worker source.
- "**A need never comes from a prompt or a checklist":** excludes idea-stage intent (v3.2 §8) and the Baseline Coverage Compiler (§9). Grounding, not origin, is the rule.
- "**Capability decides which obligations stand":** admission is trust (v3.2 §12). A plane that may call models must not decide blocking state.
- **A model floor "below which the work does not count":** breaks trust invariance.
- **Receipts invalidated when a worker's model changes:** breaks trust invariance. v3.2 §7, step 7 concerns installed providers.
- **Cross-family structure as a receipt condition:** makes a verdict depend on model identity. It belongs on the lease.
- **Barring every read of worker identity:** would kill the separation-of-duties check that independence needs.
- "**Autonomy moves exactly as far as coverage":** channels and irreversible acts are never earnable.
- **Counting every person-named need as a Felix defect:** contradicts the endgame. A person adding a need is legitimate input.
- **Auto-admitting the 22 candidates now:** every checker is flawed, and no row is armed before control 7 and the pin (roadmap Phase 1).
- **A weak corpus result means rent discovery, or narrow to obligations people name:** contradicts roadmap §5 as amended on 10-01.
- **Retiring providers keyed only on open obligations:** with today's ledger, it would propose retiring nearly every provider.
- **Holding release on every unprobed class at or above the consequence class:** would block most B2+ changes while triggers are unbuilt. v3.2 §10 says critical unknowns "can hold".
- **Scout S5's "Felix's minimum is no minimum":** misreads line 156. The real downgrade paths are lines 82 and 1237.
- **Scout S3's "now" and S6b's `← for agents` as a new control surface:** misdated, and wrong (lines 806, 41).
- **The agents view as a reason to build, or to cancel, Felix's own console work:** it is human UI and changes nothing Felix measures.

- **agentId leases as the lease guarantee:** an in-session mod hook is only a floor, and it collides with the "no plugin" lease.
- "**You should know**" **or the advisor as a judge:** invariant 2.
- **A native agent's "success" taken as a completion record:** lines 85 and 342.
- **Repo-owned, SHA-bound receipts as novel:** AI-SDLC already has them.
- **Derivation from intent as unoccupied:** Factory's 08-27 post and Missions occupy it (market map, 09-22).
- "**Every router learns from its own agent's claim of done":** no source.
- "**Projects scope one request at a time":** Projects has shared memory since 09-17.
- **The 8-of-8 cheat result as evidence that stronger models cheat better:** it measured control design, not model generations.
- **Deck slide 16 read as Felix's roadmap:** Felix governs engineering only (D2; constitution).

How this was made. Thirteen agents in one workflow. Three verifiers checked the scout's ten claims against the full changelog and swept 2.1.286 to 2.1.303 for what it missed. Four authors wrote blind from different lenses (model improvement, host absorption, founder intent, the user's experience). A fifth candidate was the main session's own draft. Each of the five had its own refuter, told to break it. All five drafts were judged repairable, and every one was caught narrowing Felix somewhere. One synthesizer then merged what survived. The main session re-read every cited changelog line and each repository figure. It corrected four of them: the commit counts, a codebook agreement figure that had no source, the corpus cost range, and the wording of v3.2 invariant 5. It also added v3.2 invariant 4 as the anchor of section 3.

Sources cited. 0cw: `docs/handoff/0cw.md`. Roadmap: `docs/2026-09-23-roadmap.md`. Constitution: `projects/felix/CLAUDE.md` in the home checkout. v3.2 and amendments: `UC Berkeley/Career/Founder OS/felix_v32_architecture_spec.md` and `felix_v32_conformance_amendments.md`. Changelog: github.com/anthropics/claude-code `CHANGELOG.md` as fetched on 2026-10-18 and cited by line. The snapshot, the deck's extracted text, the scout's report, the five candidates with their refutations, the three verification reports and the workflow script are in the memory root under `audits/2026-10-18-capability-plane/`. Memory notes are cited by name. Engine paths are under `engine/harness/`. The seeded-corpus protocol is `audits/2026-10-01-report-v5-review/seeded-corpus/protocol.md` in the memory root.